

Computer Vision Practical Report:

SHARPENING OF IMAGE USING HIGH BOOST MASK

1. AIM

To sharpen an image using High-Boost filtering in OpenCV by adding a scaled version of the high-frequency mask to the original image (or applying a high-boost spatial mask), display the results, and save the output image.

2. PROCEDURE

- Load the image using `cv2.imread()`.
- Blur the image using `cv2.GaussianBlur()` and subtract it from the original to generate the high-pass mask.
- Multiply the high-pass mask by a amplification factor $A > 1$ and add it to the original image using `cv2.addWeighted()`.
- Alternatively, apply a custom 3×3 spatial High-Boost matrix using `cv2.filter2D()`.
- Display the original and high-boost sharpened images side-by-side using Matplotlib, and save the result using `cv2.imwrite()`.

3. PROGRAM CODE

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Read the image
image = cv2.imread('E24-IMG.png')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Convert BGR to RGB

# 2. High-Boost Filter Mask (8-neighborhood, A = 1.5)
A = 1.5
kernel = np.array([
```

```
[-1, -1, -1],
[-1, A + 8, -1],
[-1, -1, -1]
], dtype=np.float32)

# 3. Apply Sharpening Filter
sharpened = cv2.filter2D(image, -1, kernel)
sharpened_rgb = cv2.cvtColor(sharpened, cv2.COLOR_BGR2RGB)

# 4. Plot both images on a single page/figure
plt.figure(figsize=(10, 5))

# Original Image
plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title('Original Image')
plt.axis('off')

# Sharpened Output Image
plt.subplot(1, 2, 2)
plt.imshow(sharpened_rgb)
plt.title('High-Boost Sharpened Output')
plt.axis('off')

plt.tight_layout()
plt.show()
```

4. OUTPUT

Original Image



High-Boost Sharpened Output



5. RESULT

The Python script executed successfully. The input image was loaded.